



Selenium 4 New Features

FEB,2026

(expleo)

Outline

- Enhanced Selenium Grid
- Simplification to open a new Windows browser and Tabs
- Relative Locators
- TakeElementScreenshot
- New additions to the Actions Class



Enhanced Selenium Grid



Selenium Grid and Modes

- Managing Selenium Grid is now smooth and easy as there will no longer be any need to set up and start hubs and nodes separately. The grid can be deployed in 3 modes:

Standalone – Standalone is the union of all components, and to the user's eyes, they are executed as one.

- A fully functional Grid of one is available after starting it in the Standalone mode. By default, the server will be listening on `http://localhost:4444`, and that's the URL you should point your RemoteWebDriver tests.
- The server will detect the available drivers that it can use from the System PATH.

Selenium Grid and Modes

Hub and Node – It enables the classic Hub & Node(s) setup. These roles are suitable for small and middle-sized Grids

Distributed – On Distributed mode, each component needs to be started on its own. This setup is more suitable for large Grids.

- Grid will now support IPv6 addresses and one can communicate with the Grid using the HTTPS protocol. The new Selenium Grid comes with Docker support. It also supports advanced tools like AWS, Azure, and much more, useful in the DevOps process. Now Grid has a more user-friendly UI and contains relevant information related to the session,

running, capacity, etc.

Simplification to open a new Windows browser and Tabs



Overview

- There are a number of scenarios where you would want to open a new browser (or tab) and perform a certain set of actions in the newly opened window/tab.
- In Selenium 3, you have to create a new Web Driver object and then switch to the new window (or tab) using its unique WindowHandle to perform subsequent actions in that window (or tab).

Windows browser and Tabs

- Selenium 4 provides a new API `new Window` that lets you create a new window (or tab) and automatically switch to it.
- Since the new window or tab is created in the same session, it avoids creating a new `WebDriver` object.
- For switching to the new tab, pass `WindowType.TAB` to `newWindow()` method and for creating a new window, pass `WindowType.WINDOW` to `newWindow()` method.

Simplification to open a new Windows browser and Tabs

Selenium Code

```
public class NewWindowDemo {  
  
    public static void main(String[] args) {  
        WebDriver driver = new ChromeDriver();  
        driver.manage().window().maximize();  
  
        driver.get("https://www.bing.com/");  
        System.out.println("New Page - Bing is opened");  
  
        // Opens a new window and switches to new window  
        driver.switchTo().newWindow(WindowType.WINDOW);  
  
        // Opens duckduckgo homepage in the new opened window  
        driver.navigate().to("https://www.duckduckgo.com/");  
  
        System.out.println("New Page - DuckDuckGo is opened");  
  
        driver.quit();  
    }  
}
```

Simplification to open a new Windows browser and Tabs

Selenium Code

```
public class NewTabDemo {  
  
    public static void main(String[] args) {  
  
        WebDriver driver = new ChromeDriver();  
        driver.manage().window().maximize();  
  
        driver.get("https://www.bing.com/");  
        System.out.println("New Page - Bing is opened");  
  
        // Opens a new window and switches to new window  
        driver.switchTo().newWindow(WindowType.TAB);  
  
        // Opens duckduckgo homepage in the new opened window  
        driver.navigate().to("https://www.duckduckgo.com/");  
  
        System.out.println("New Tab is opened with DuckDuckGo");  
        driver.quit();  
    }  
}
```

Relative Locators



Overview

- Selenium 4 brings Relative Locators which are previously called as Friendly Locators. This functionality was added to help you locate elements that are nearby other elements.
- The Available Relative Locators are:
 - a) above
 - b) below
 - c) toLeftOf
 - d) toRightOf
 - e) near

Relative Locators

- **findElement** method now accepts a new method **withTagName()** which returns a RelativeLocator.

```
import static org.openqa.selenium.support.locators.RelativeLocator.with;
```

a) above() – Returns the WebElement, which appears above the specified element.

```
WebElement passwordField_above = driver.findElement(By.id("txtPassword"));
```

```
WebElement emailAddressField_above =
```

```
driver.findElement(with(By.tagName("input")).above(passwordField_above));
```

Relative Locators

b) below() – Returns the WebElement, which appears below the specified element.

```
WebElement emailAddress_below = driver.findElement(By.id("txtUsername"));
```

```
WebElement passwordField_below =
```

```
driver.findElement(with(By.tagName("input")).below(emailAddress_below));
```

Relative Locators

c) toRightOf() – Target web element which is presented to the right of a specified element.

```
WebElement submitButton= driver.findElement(By.id("submit"));
```

```
WebElement cancelButton= driver.findElement(with(By.tagName("button"))  
                                             .toRightOf(submitButton));
```

Relative Locators

d) toLeftOf() – Returns the WebElement, which appears to the left of the specified element.

```
WebElement cancelButton= driver.findElement(By.id("cancel"));
```

```
WebElement submitButton=
```

```
driver.findElement(with(By.tagName("button")).toLeftOf(cancelButton));
```


Relative Locators

e) near() – Returns the WebElement, which is at most 50px away from the specified element.

```
WebElement emailAddressLabel= driver.findElement(By.id("lbl-email"));
```

```
WebElement emailAddressField =
```

```
driver.findElement(with(By.tagName("input")).near(emailAddressLabel));
```

TakeElement Screenshot



Overview

- In Selenium 3, there was a provision to capture a screenshot of the entire web page.
- Selenium 4 onwards, there is a new option to capture screenshots of a particular WebElement. Hence, there is no need to use third-party tools like Shutterbug, Ashot, etc. (like in Selenium 3) for capturing a screenshot of WebElement.
- The newly introduced method in Selenium 4 captures the screenshot of an element for the current browsing context.
- The screenshot returned by the WebDriver endpoint is encoded in the Base64 format.

Taking screenshots

Introduction

- Taking screenshots during automated testing is an essential practice for debugging and reporting purposes.
- Selenium WebDriver provides built-in functionality to capture screenshots of web pages at any point in the test execution.
- Here are some typical cases where its crucial to take screenshot in Selenium:
 - When application issues occur
 - When an assertion failure occurs
 - When there is some difficulty in finding web elements on a page
 - Where there is a Timeout in finding web elements on a web page

Taking screenshots

Prerequisites for taking a Screenshot in Selenium

- Taking screenshots in Selenium requires a few basic tools and steps to be set up properly.

Here's what you will need to get started:

1. Selenium WebDriver: The core tool for automating browser actions. It allows you to perform tasks like clicking buttons, filling forms, and of course, capturing screenshots. Without Selenium WebDriver, taking a screenshot in Selenium isn't possible.

2. TakesScreenshot Interface: The TakesScreenshot interface is a special feature in Selenium that enables you to take a screenshot of your current browser window. By using its `getScreenshotAs()` method, you can capture an image in different formats like a file or base64.

3. OutputType Class: To define how the screenshot should be saved, you'll use the `OutputType` class. The most common usage is `OutputType.FILE`, which saves the screenshot as a file, but you can also save it in base64 format if needed.

Taking screenshots

Prerequisites for taking a Screenshot in Selenium

4. Java File Handling: To save the screenshot, you will need to work with Java's file handling features. The File class is used to define the screenshot, and FileUtils (from Apache Commons IO) helps in saving it to a specific location.

5. Appropriate Browser Driver: Ensure you have the right browser driver, such as ChromeDriver or GeckoDriver, installed for your browser.

- Once you have these prerequisites in place, capturing screenshots in Selenium becomes a simple task.

Page Screenshot

- To capture a screenshot in Selenium, we can make use of an interface, called **TakesScreenshot**. This method indicates the driver, that it can capture a screenshot and store it in different ways

```
TakesScreenshot ts = (TakesScreenshot) driver;
```

- In order to capture screenshot and store it in a particular location, there is a method called "**getScreenshotAs**", where `OutputType` defines the output type for a screenshot.

```
File source = ts.getScreenshotAs(OutputType.FILE);
```

Page Screenshot

- Copy file to Desired Location

```
FileUtils.copyFile(source, newFile("./Screenshots/Selenium" + System.currentTimeMillis()  
+ ".png"));
```


Element - Screenshot

1. Take the screenshot of the element:

- To capture a screenshot in Selenium, we can make use of an interface, called TakesScreenshot. This method indicates the driver, that it can capture a screenshot and store it in different ways

File source = logo.getScreenshotAs(OutputType.FILE);

- In order to capture screenshot and store it in a particular location, there is a method called "getScreenshotAs", where OutputType defines the output type for a screenshot.

Element - Screenshot

2. Specify the location to save the screenshot:

File screenshotLocation = new

File("src//test/resources//screenshot/specific_element_screenshot.png");

3. Save the screenshot to the specified location:

We want to save the element in the screenshot folder present in src/test/resources directory. So, copy the screenshot in that folder.

FileUtils.copyFile(source, screenshotLocation);

Taking screenshots

Taking screenshots - Example

```
import java.io.IOException;
import java.io.File;
import org.apache.commons.io.FileUtils;
import org.openqa.selenium.TakesScreenshot;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class SeleniumScreen{
public static void main(String[] args) throws Exception {
// Instantiate chrome driver object
WebDriver driver = new ChromeDriver();
// Navigate to the testim website
driver.get("https://www.testim.io/");
//Use TakesScreenshot method to capture screenshot
TakesScreenshot screenshot = (TakesScreenshot)driver;
```

Taking screenshots

Taking screenshots - Example

```
//Saving the screenshot in desired location
File source = screenshot.getScreenshotAs(OutputType.FILE);
//Path to the location to save screenshot
FileUtils.copyFile(source, new File("./SeleniumScreenshots/Screen.png"));
System.out.println("Screenshot is captured");
driver.quit();
}
}
```

Simplification to open a new Windows browser and Tabs

Selenium Code

```
import org.apache.commons.io.FileUtils;
import org.openqa.selenium.By;
import org.openqa.selenium.OutputType;
import org.openqa.selenium.TakesScreenshot;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.firefox.FirefoxDriver;

public class ScreenshotDemo {

    public static void main(String[] args) {
        WebDriver driver = new ChromeDriver();
        driver.manage().window().maximize();
        driver.get("https://www.selenium.dev/");

        try {
            WebElement logo = driver.findElement(By.xpath("//*[@id='td-cover-block-0']/div/div/div/div/h1"));
            File source = ((TakesScreenshot) logo).getScreenshotAs(OutputType.FILE);
            FileUtils.copyFile(source, new File("./Screenshots/logo" + System.currentTimeMillis() + ".png"));
        }
    }
}
```

Simplification to open a new Windows browser and Tabs

Selenium Code

```
catch (Exception e) {  
    System.out.println(e.getMessage());  
}  
  
System.out.println("The Screenshot is taken and saved under Screenshots folder");  
driver.quit();  
}  
}
```

Selenium Code

```
import org.apache.commons.io.FileUtils;  
import org.openqa.selenium.By;  
import org.openqa.selenium.OutputType;  
import org.openqa.selenium.TakesScreenshot;  
import org.openqa.selenium.WebDriver;  
import org.openqa.selenium.WebElement;  
import org.openqa.selenium.chrome.ChromeDriver;  
import org.openqa.selenium.firefox.FirefoxDriver;  
  
public class ScreenshotDemo {  
  
    public static void main(String[] args) {  
        FirefoxOptions options = new FirefoxOptions();  
        WebDriver driver = new FirefoxDriver(options);  
        driver.manage().window().maximize();  
        driver.get("https://www.selenium.dev/");  
    }  
}
```

Simplification to open a new Windows browser and Tabs

Selenium Code

```
// Locate the specific element
WebElement logo = driver.findElement(By.xpath("//div[@class='row']/div"));

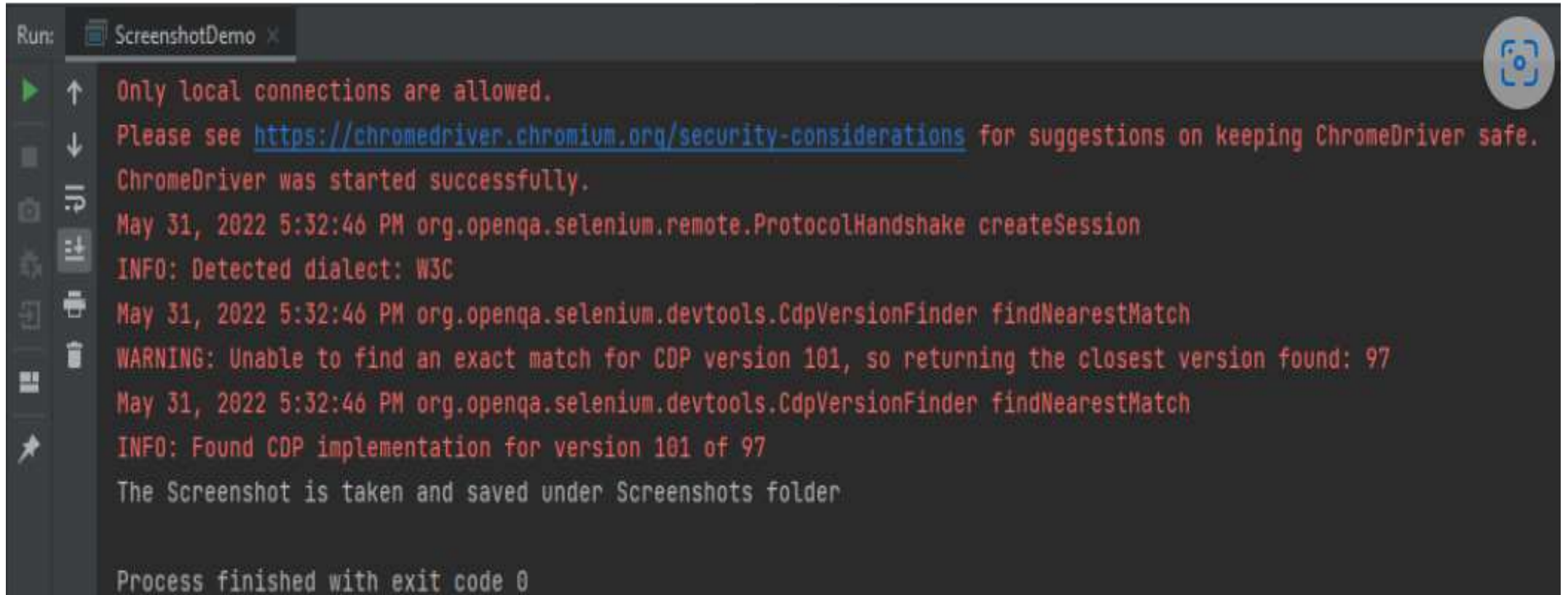
// Take the screenshot of the element
File source = logo.getScreenshotAs(OutputType.FILE);

// Specify the location to save the screenshot
File screenshotLocation = new File("src//test/resources//screenshot/element_screenshot.png");

try {
    // Save the screenshot to the specified location
    FileUtils.copyFile(source, screenshotLocation);
    System.out.println("Screenshot saved to: " + screenshotLocation.getAbsolutePath());
} catch (Exception e) {
    System.out.println(e.getMessage());
}

System.out.println("The Screenshot is taken and saved under Screenshots folder");
driver.quit();
}
```


Output

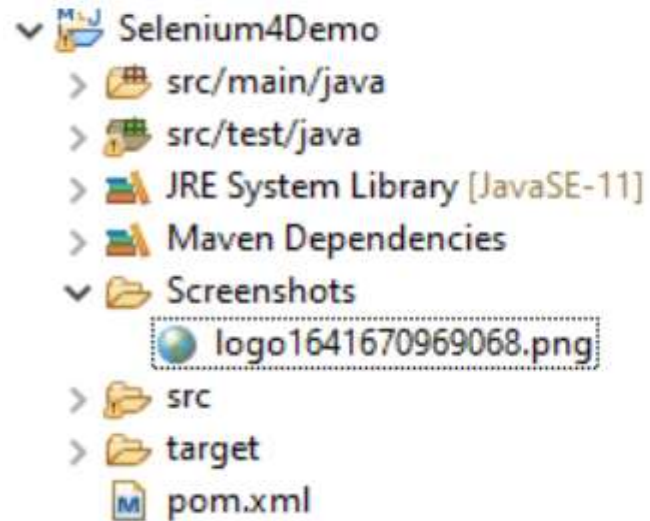


```
Run: ScreenshotDemo x
Only local connections are allowed.
Please see https://chromedriver.chromium.org/security-considerations for suggestions on keeping ChromeDriver safe.
ChromeDriver was started successfully.
May 31, 2022 5:32:46 PM org.openqa.selenium.remote.ProtocolHandshake createSession
INFO: Detected dialect: W3C
May 31, 2022 5:32:46 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find an exact match for CDP version 101, so returning the closest version found: 97
May 31, 2022 5:32:46 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
INFO: Found CDP implementation for version 101 of 97
The Screenshot is taken and saved under Screenshots folder

Process finished with exit code 0
```

Handling Browser Navigation

Output - The picture will be saved in the Screenshots folder as shown below:



Below is the image of the screenshots.



New additions to the Actions Class



Overview

- Actions Class in Selenium provides several methods for performing a single action or a series of actions on the WebElements present in the DOM.
- Mouse actions (e.g., click, double click, etc.) and Keyboard actions (e.g., keyUp, keyDown, sendKeys) are the two broad categories of Actions.
- For demonstration, we will post the examples demonstrated in the Action class in the Selenium blog from Selenium 3 to Selenium 4.

Action Class – New Methods

- With Selenium 4, new methods are added to the Actions class, which replaces the classes under the `org.openqa.selenium.interactions` package.

1. `click(WebElement)` is the new method added to the Actions class and it serves as the replacement of **`moveToElement(onElement).click()`** method.

- Like the method in the versions before Selenium 4, `click(WebElement)` is used for clicking a web element.

2. `doubleClick(WebElement)` - This method is added to replace **`moveToElement(element).doubleClick()`**. It will perform a double-click on an element.

Selenium Code

```
import org.openqa.selenium.Alert;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.interactions.Actions;

public class DoubleClick {
    public static void main(String[] args) {

        WebDriver driver = new ChromeDriver();

        // Navigate to Url
        driver.get("https://demo.guru99.com/test/simple_context_menu.html");

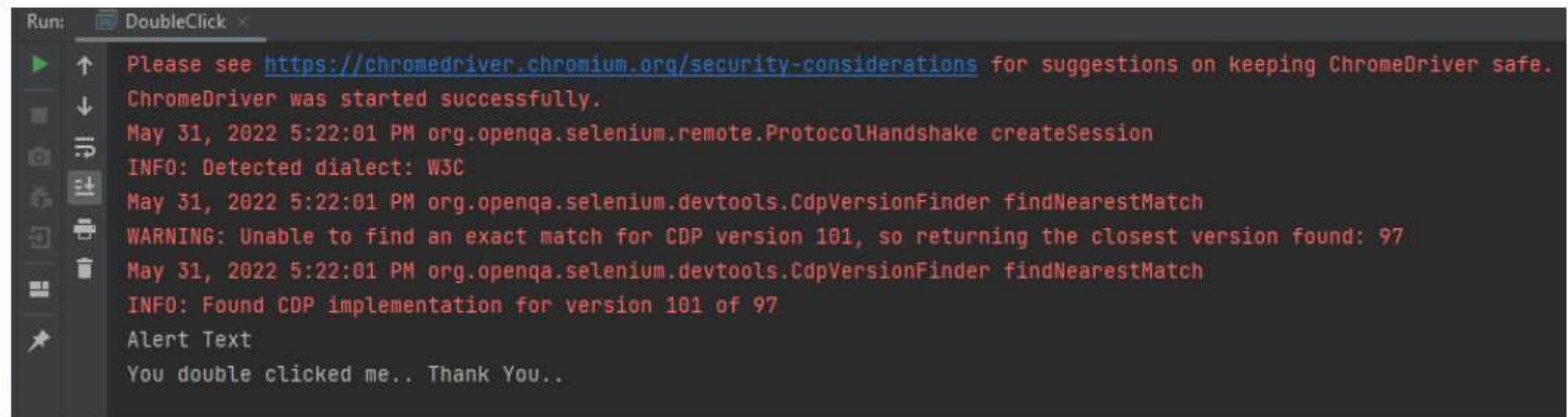
        // Store 'doubleClickButton' button web element
        WebElement doubleClickButton = driver.findElement(By.xpath("//*[@id='authentication']/button"));
        Actions actionProvider = new Actions(driver);
```


New additions to the Actions Class

Selenium Code

```
// Perform double-click action on the element
actionProvider.doubleClick(doubleClickButton).build().perform();
Alert alert = driver.switchTo().alert();
System.out.println("Alert Text\n" +alert.getText());
alert.accept();
driver.close();
}
```

The output of the above program is



```
Run: DoubleClick x
Please see https://chromedriver.chromium.org/security-considerations for suggestions on keeping ChromeDriver safe.
ChromeDriver was started successfully.
May 31, 2022 5:22:01 PM org.openqa.selenium.remote.ProtocolHandshake createSession
INFO: Detected dialect: W3C
May 31, 2022 5:22:01 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find an exact match for CDP version 101, so returning the closest version found: 97
May 31, 2022 5:22:01 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
INFO: Found CDP implementation for version 101 of 97
Alert Text
You double clicked me.. Thank You..
```

Action Class – New Methods

3.clickAndHold(WebElement)

- This method will replace the `moveToElement(onElement).clickAndHold()`. It is used to click on an element without releasing the click.

4.contextClick(WebElement)

- This method will replace `moveToElement(onElement).contextClick()`. It will perform the right-click operation.

5. release()

- This method (user for releasing the pressed mouse button) was initially a part of `org.openqa.selenium.interactions.ButtonReleaseAction` class.

Now with the updated version of Selenium, it has been moved to the Actions class.

Selenium Code

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.interactions.Actions;

public class clickAndHold {
    public static void main(String[] args) {

        WebDriverManager.chromedriver().setup();
        WebDriver driver = new ChromeDriver();

        // Navigate to Url
        driver.get("https://crossbrowsertesting.github.io/drag-and-drop.html");
        driver.manage().window().maximize();

        // Find element xpath which we need to drag
        WebElement from = driver.findElement(By.id("draggable"));
```

Selenium Code

```
// Find element xpath where we need to drop  
WebElement to = driver.findElement(By.id("droppable"));
```

```
Actions actionProvider = new Actions(driver);
```

```
// Perform click-and-hold action on the element  
actionProvider.clickAndHold(from).build().perform();
```

```
// Move to drop WebElement  
actionProvider.clickAndHold(to).build().perform();
```

```
//Release drop element  
actionProvider.release(to).build().perform();
```

```
}
```

```
}
```

Output

